HaloFormCraft

Technical Documentation

Current Application & System Specification

Generated for Internal Audit - May 2026

HaloFormCraft: Current Technical Specification

1. Project Overview

HaloFormCraft is a schema-driven form orchestration platform currently optimized for clinical and healthcare data collection.

1.1 Current Application Scope

- **Dynamic Orchestration:** Interprets JSON schemas to render complex, multi-step forms.
- **Clinical Logic:** Automated calculations for BMI, scores, and age.
- **Status Management:** Supports Draft and Published versions of forms.
- **Export Capabilities:** CSV data extraction for all form submissions.

1.2 Key Implemented Features

- **Visual Form Builder:** Drag-and-drop orchestration with 20+ field types (Text, Choices, Health-specific).
- **Exclusive Stop Criteria:** Dynamic "slicing" of form sections based on critical input triggers.
- **Specialized Inputs:** Integrated GPS, Signature capture, and QR scanning components.
- **Conditional Visibility:** Real-time hide/show logic based on user answers.

---

2. System Architecture

2.1 Overall Architecture

The system is built as a **Decoupled Client-Server Architecture**.

2.2 Frontend (React SPA)

- **Rendering Engine:** interpreters JSON schemas into interactive UI.
- **State Management:** React Hook Form for validation and value tracking.
- **Build Tool:** Powered by Vite for high-speed development and optimized production builds.

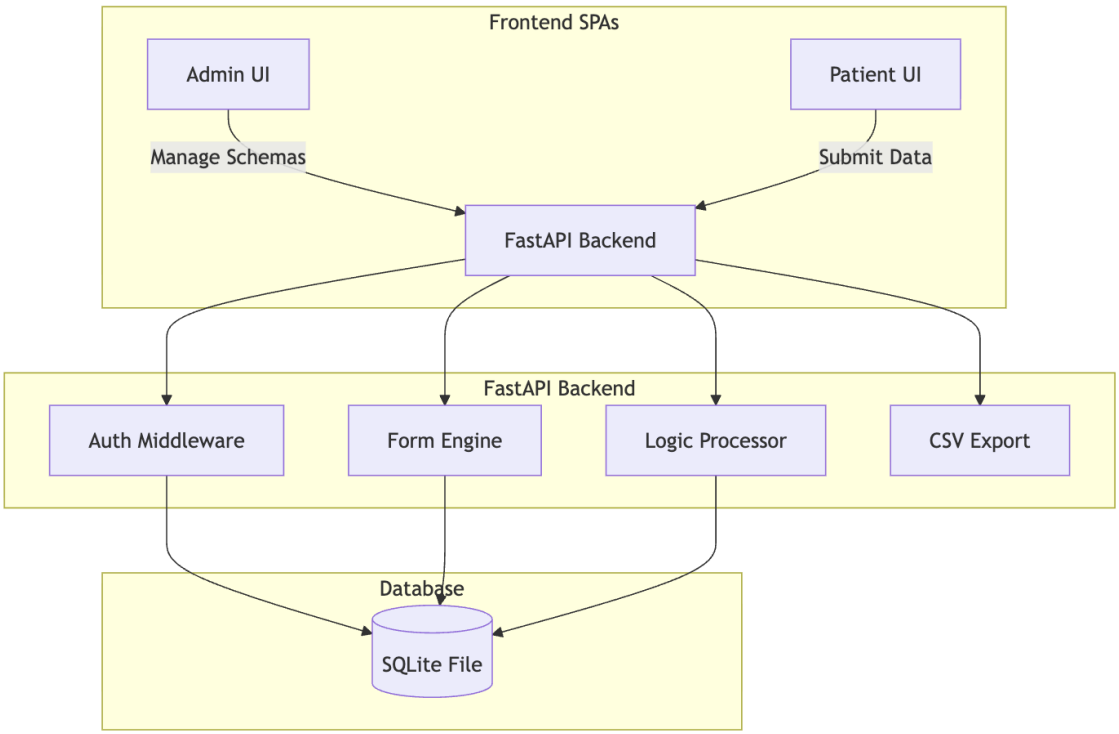
2.3 Backend (FastAPI)

- **ASGI Framework:** High-performance asynchronous API layer.
- **RESTful Endpoints:** Standardized CRUD operations for form management and data submission.

2.4 Persistence Layer

- **Primary Database:** SQLite (local file-based storage).
- **ORM:** SQLAlchemy for relational mapping.

12.1 System Architecture Overview



3. Tech Stack

Layer	Technology	Status
Frontend	React 19, Vite, Tailwind CSS 4	Active
Logic/Validation	React Hook Form, Yup	Active
Specialized UI	Leaflet, html5-qrcode, signature-canvas	Active
Backend	FastAPI, SQLAlchemy, Uvicorn	Active
Database	SQLite	Active
Icons	Lucide-React	Active

4. Folder Structure

```
/form-creation
|-- /admin-ui          # Form Builder and Analytics
|   |-- /src/components/builder   # Canvas, Toolbox, Settings
|   |-- /src/pages                # Dashboard, Builder, Responses
|-- /patient-ui        # Data Collection Portal
|   |-- /src/App.jsx              # Rendering engine and submission logic
|-- /backend           # API Service
|-- main.py                  # FastAPI Routers
|-- models.py                # SQLAlchemy Models
|-- database.py              # DB Connection
|-- schemas.py               # Pydantic Schemas
```

5. Features Documentation

5.1 Exclusive Stop Criteria

- **Purpose:** Terminates form flow immediately upon specific clinical triggers.
- **Workflow:** If a marked option is selected, the engine truncates subsequent pages and forces a "Submit" state.

5.2 BMI & Calculated Scores

- **BMI:** Updates based on Height/Weight field IDs using reactive hooks.
- **Scoring:** Aggregates `optionScores` from choice-based fields to produce a total value.

6. Database Design

6.1 Implemented Tables

- **forms:** Master metadata for each form.
- **form_versions:** Stores the JSON `schema` and `status` (DRAFT/PUBLISHED).
- **submissions:** Relational store for submission data payloads.
- **users:** Basic role-based identity (ADMIN/PATIENT).

7. API Documentation

Endpoint	Method	Role	Purpose
`/api/admin/forms`	GET	Admin	List all forms
`/api/admin/forms/{id}/publish`	POST	Admin	Move Draft to Published
`/api/patient/forms/{id}`	GET	Patient	Load published schema
`/api/patient/forms/{id}/submissions`	POST	Patient	Record user responses

8. Authentication & Security

- **RBAC:** Mock role-based headers used for development.
- **Validation:** Pydantic models enforce schema integrity on all POST requests.
- **CORS:** Configured for cross-origin local development.

9. Scalability & Performance

- **Asynchronous IO:** Backend uses `async` handlers to prevent blocking.
- **Idempotency:** UUID-based tracking for submissions.

- **Statelessness:** API can be horizontally scaled with multiple workers.

10. Error Handling

- **Frontend:** Toast notifications and inline field errors.
- **Backend:** HTTP 404/400 exceptions with descriptive detail messages.

11. Screens & UI Documentation

- **Dashboard:** Management grid with Status indicators.
- **Builder:** Drag-and-drop workspace with real-time Preview toggle.
- **Patient Form:** Responsive, step-by-step navigation with Progress Bar.

12. Diagrams & Technical Visuals

12.1 Current Submission Flow

The following diagram illustrates the direct data transmission from the Patient UI to the persistent storage layer via the FastAPI backend.